



数据结构

Data Structure

许 蕾

xlei@nju.edu.cn

南京大学计算机学院



第十章 文件、外部排序 与外部搜索



- 主存储器和外存储器
- 文件组织
- 多级索引结构
- 外排序



4 倒排表 (Inverted Index List)



- 对包含有大量数据记录的数据表或文件进行搜索时，最常用的是针对记录的主关键码建立索引。主关键码可以唯一地标识该记录。用主关键码建立的索引叫做**主索引**。
- 主索引的每个索引项给出记录的关键码和记录在表或文件中的存放地址。

记录关键码 <i>key</i>	记录存放地址 <i>addr</i>
------------------	--------------------

- 倒排：将传统的数据组织方向（记录→属性）反转过来，建立属性→记录的映射，以支持基于属性值的高效查询。

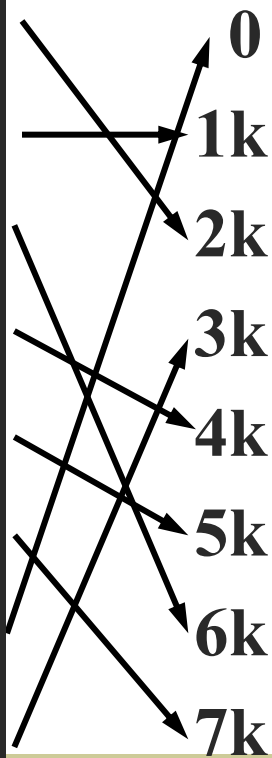


- 在实际应用中，有时需要针对**其它属性**进行搜索。例如，查询如下的职工信息：
 - (1) 列出所有教师的名单；
 - (2) 已婚的女性职工有哪些人？
- 这些信息在数据表或文件中都存在，但都不是关键码，为回答以上问题，只能到表或文件中去顺序搜索，搜索效率极低。
- 因此，除主关键码外，可以把一些经常搜索的属性设定为**次关键码**，并**针对每一个作为次关键码的属性，建立次索引**。
- 在次索引中，列出该属性的所有取值，并对每一个取值建立有序链表，把所有具有相同属性值的记录**按存放地址递增的顺序**或**按主关键码递增的顺序**链接在一起。



索引表

key	addr
03	2k
08	1k
17	6k
24	4k
47	5k
51	7k
83	0
95	3k



数据表

职工号	姓名	性别	职务	婚否	...
83	张珊	女	教师	已婚	...
08	李斯	男	教师	已婚	...
03	王璐	男	教务员	已婚	...
95	刘琪	女	实验员	未婚	...
24	岳跋	男	教师	已婚	...
47	周斌	男	教师	已婚	...
17	胡江	男	实验员	未婚	...
51	林青	女	教师	未婚	...



索引非顺序文件示例



- 次索引的索引项由次关键码、链表长度和链表本身等三部分组成。
- 例如，为了回答上述的查询，我们可以分别建立“性别”、“婚否”和“职务”次索引。

性别次索引

次关键码	男	女
计 数	5	3
地址指针		

指针	03	08	17	24	47	51	83	95
----	----	----	----	----	----	----	----	----



婚否次索引

次关键码	已婚	未婚
计 数	5	3
地址指针		

指针	03	08	24	47	83	17	51	95
----	----	----	----	----	----	----	----	----

职务次索引

次关键码	教师	教务员	实验员
计 数	5	1	2
地址指针			

指针	08	24	47	51	83	03	17	95
----	----	----	----	----	----	----	----	----





- (1) 列出所有教师的名单;
- (2) 已婚的女性职工有哪些人?

- 通过顺序访问“职务”次索引中的“教师”链，可以回答上面的查询(1)。
- 通过对“性别”和“婚否”次索引中的“女性”链和“已婚”链进行求“交”运算，就能够找到所有既是女性又是已婚的职工记录，从而回答上面的查询(2)。
- 倒排表是次索引的一种实现。在表中所有次关键码的链都保存在次索引中，仅通过搜索次索引就能找到所有具有相同属性值的记录。
- 在次索引中记录记录存放位置的指针可以用主关键码表示：可通过搜索次索引确定该记录的主关键码，再通过搜索主索引确定记录的存放地址。



- 在倒排表中各个属性链表的长度大小不一, 管理比较困难。为此引入单元式倒排表。
- 在单元式倒排表中, 索引项中不存放记录的存储地址, 而是存放该记录所在硬件区域 (即存储区域) 的标识。
- 硬件区域可以是磁盘柱面、磁道或一个页块, 以一次 I/O 操作能存取的存储空间作为硬件区域为最好。



- 为使索引空间最小，在索引中标识这个硬件区域时可以使用一个能转换成地址的二进制数，整个次索引形成一个(二进制数的)位矩阵。
- 例如，对于记录学生信息的文件，次索引可以是如图（下页）所示的结构。二进制位的值为 1 的硬件区域包含具有该次关键码的记录。



		硬 件 区 域										
		1	2	3	4	5	...	251	252	253	254	...
次关键码 1 (性别)	男	1	0	1	1	1	...	1	0	1	1	...
	女	1	1	1	1	1	...	0	1	1	0	...
次关键码 2 (籍贯)	广东	1	0	0	1	0	...	0	1	0	0	...
	北京	1	1	1	1	1	...	0	0	1	1	...
	上海	0	0	1	1	1	...	1	1	0	0	...
												
次关键码 3 (专业)	建筑	1	1	0	0	1	...	0	1	0	1	...
	计算机	0	0	1	1	1	...	0	0	1	1	...
	电机	1	0	1	1	0	...	1	0	1	0	...
												

单元式倒排表结构



- 针对一个查询：找出所有广东籍学建筑的男学生。可以从“性别”、“籍贯”、“专业”三个次索引分别抽取属性值为“男”、“广东”、“建筑”的位向量，按位求交，求得满足查询要求的记录在哪些硬件区域中，再读入这些硬件区域，从中查找所需的数据记录。

	1	0	1	1	1		1	0	1	1
	1	0	0	1	0		0	1	0	0
AND	1	1	0	0	1		0	1	0	1
	1	0	0	0	0		0	0	0	0

- 由运算结果可知，在硬件区域1，.....中有所需的记录。然后将硬件区域1，.....等读入内存，在其中进行检索，就可取出所需记录。



记录号	姓 名	学 号		专 业		已修学分		选 修 课 目					
01	王 雯	1350	02	软件	02	412	03	丙	02	丁	03	丁	05
02	马小燕	1351	03	软件	07	398	07	甲	04	丙	03		
03	阮 森	1352	04	计算机	05	436	Λ	乙	05	丙	04		
04	苏明明	1353	Λ	应用	06	402	08	甲	06	丙	08		
05	田 永	1354	06	计算机	Λ	384	02	乙	07	丁	09		
06	杨 青	1355	07	应用	09	356	10	甲	07				
07	薛平平	1356	08	软件	08	398	Λ	甲	08	乙	Λ		
08	崔子健	1357	Λ	软件	Λ	408	01	甲	09	丙	Λ		
09	王 洪	1358	10	应用	10	370	05	甲	10	丁	Λ		
10	刘 倩	1359	Λ	应用	Λ	364	09	甲	Λ				

软 件	01, 02, 07, 08
计 算 机	03, 05
应 用	04, 06, 09, 10

(a)专业倒排表

350~399	02, 05, 06, 07, 09, 10
400~449	01, 03, 04, 08

(b)已修学分倒排表

甲	02, 04, 06, 07, 08, 09, 10
乙	03, 05, 07,
丙	01, 02, 03, 04, 08
丁	01, 03, 05, 09

(c)选修课目倒排表



1. 关于顺序文件的描述，正确的是：

- A) 适合频繁插入和删除记录的操作
- B) 只能采用顺序存取方式访问
- C) 存储在磁带上的文件通常是顺序文件
- D) 查找某个特定记录时效率最高

■ 正确答案： C

■ 解析： 顺序文件中的记录按逻辑顺序连续存储，磁带由于物理特性只能顺序访问，因此非常适合存储顺序文件。顺序文件可以采用顺序或随机存取（如果支持），但不适合频繁插入删除，查找效率也不是最高。



2. 在索引顺序文件中，关于溢出区的使用，正确的是：

- A) 溢出区记录不参与索引
 - B) 溢出区采用顺序组织方式
 - C) 主数据区记录删除时，用溢出区记录填补
 - D) 溢出区目的是减少索引大小
-
- 正确答案： A
 - 解析： 索引顺序文件中，溢出区存放新增记录，这些记录不包含在主索引中，需要通过指针链访问。溢出区通常采用链表组织，而不是顺序组织。



- 1. 在直接（散列）文件组织中，记录通过 _____ 函数转换为存储地址。当不同键值映射到同一地址时，称为 _____，需要通过 _____ 方法解决。
- 正确答案： 散列（哈希）、冲突（碰撞）、冲突处理（如开放寻址、链地址法）
- 2. 倒排文件的主要特点是建立了 _____ 到 _____ 的索引，特别适合 _____ 查询。
- 正确答案： 属性值（字段值）、记录标识（记录地址）、多条件组合（或基于属性的）



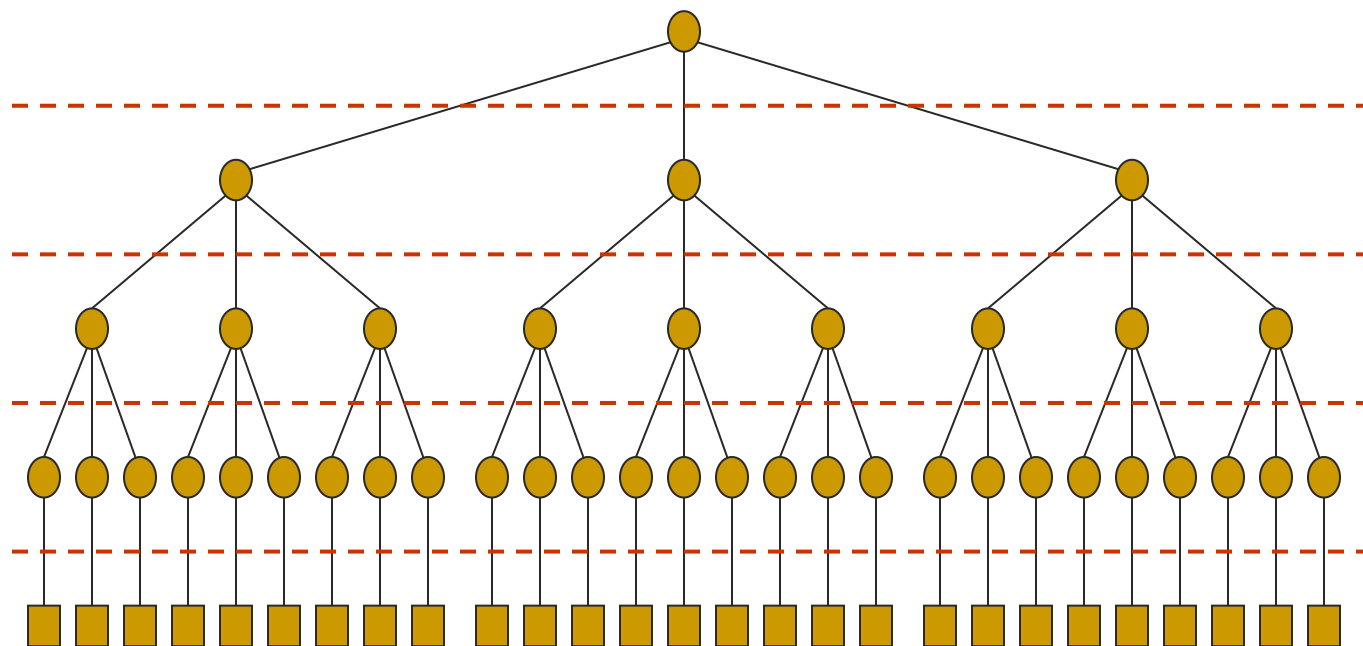
10.4 多级索引结构



- 当数据记录数目特别大，索引表本身也很大，在内存中放不下，需要分批多次读取外存才能把索引表搜索一遍。
- 此时，可以建立索引的索引(二级索引)。二级索引可以常驻内存，二级索引中一个索引项对应一个索引块，登记该索引块的**最大关键码**及该索引块的**存储地址**。
- 如果二级索引在内存中也放不下，需要分为许多块多次从外存读入。可以建立二级索引的索引(三级索引)。这时，访问外存次数等于读入索引次数再加上1次读取记录。



- 必要时, 还可以有4级索引, 5级索引, ...。
- 多级索引结构常用 m 叉树表示, 称为 m 路搜索树。
- m 路搜索树可能是静态索引结构, 即结构在初始创建, 数据装入时就已经定型, 在整个运行期间, 树的结构不发生变化。
- m 路搜索树还可能是动态索引结构, 即在整个系统运行期间, 树的结构随数据的增删及时调整, 以保持最佳的搜索效率。



四级索引

三级索引

二级索引

一级索引

数据区

多级索引结构形成 m 路搜索树



10.4.1 ISAM (索引顺序存取方法文件)



- ISAM是静态索引结构。典型的例子是对磁盘上的数据文件建立盘组、柱面、磁道三级地址的多级索引。
- ISAM文件用柱面索引对各个柱面进行索引。一个柱面索引项保存该柱面上的最大关键码（最后一个记录）以及柱面开始地址指针。
- 如果柱面太多，可以建立柱面索引的分块索引，即主索引。
- 主索引不太大，一般常驻内存。



主索引

525	C ₀ T ₁
1510	C ₆ T ₁
...	
...	
...	
5540	C ₇₉ T ₁

柱面索引

125	C ₀ T ₀
270	C ₁ T ₀
...	
525	C ₅ T ₀
714	C ₆ T ₀
833	C ₇ T ₀
...	
1510	C ₁₁ T ₀
...	
...	
5540	C ₈₄ T ₀

各柱面信息

C₀T₀
C₀T₁
C₀T₂
C₀T₃
C₀T₄
.....

55	C ₀ T ₁		100	C ₀ T ₂		125	C ₀ T ₃	
R ₂₀	R ₂₅	R ₃₀	R ₄₀	R ₄₅	R ₄₈	R ₅₅		
R ₆₄	R ₆₉	R ₇₄	R ₇₈	R ₈₃	R ₉₁	R ₁₀₀		
R ₁₁₀	R ₁₂₅							
溢出区								

C₁T₀
C₁T₁
C₁T₂
C₁T₃
C₁T₄
.....

181	C ₁ T ₁		222	C ₁ T ₂		270	C ₁ T ₃	
R ₁₄₆	R ₁₅₁	R ₁₅₉	R ₁₆₄	R ₁₆₈	R ₁₇₂	R ₁₈₁		
R ₁₈₉	R ₁₉₀	R ₁₉₃	R ₁₉₈	R ₂₀₃	R ₂₁₀	R ₂₂₂		
R ₂₃₄	R ₂₄₆	R ₂₅₅	R ₂₆₉	R ₂₇₀				
溢出区								

C₇T₀
C₇T₁
C₇T₂
C₇T₃
C₇T₄
.....

758	C ₇ T ₁		793	C ₇ T ₂		833	C ₇ T ₃	
R ₇₂₀	R ₇₂₄	R ₇₂₇	R ₇₃₆	R ₇₄₃	R ₇₅₉	R ₇₅₈		
R ₇₆₅	R ₇₆₉	R ₇₇₇	R ₇₈₁	R ₇₈₅	R ₇₉₀	R ₇₉₃		
R ₇₉₉	R ₈₀₁	R ₈₂₅	R ₈₃₃					
溢出区								

磁道索引

磁道索引

磁道索引



- 在每个柱面上，所有数据记录存放于基本区，此外保留一部分磁道作为溢出区。所有记录在基本区按关键码升序排列，后一磁道所有记录的关键码均大于前一磁道所有记录的关键码。
- 在一个柱面上所有记录分布在一系列磁道上，通过**磁道索引**进行搜索。磁道索引一般放在每个柱面上第0号磁道中。
- 每个磁道索引的索引项由两部分组成：

最大关键码	开始地址指针	最大关键码	溢出链头指针
-------	--------	-------	--------

基本区索引项

溢出区索引项



- 基本区索引项存放本磁道在基本区最大关键码（在基本区该磁道最后一个记录）和本磁道在基本区的开始地址。
- 溢出区索引项存放本磁道在溢出区最大关键码和本磁道在溢出区中溢出记录链（有序链表）的第一个结点地址。
- 在某一磁道插入一个新记录时，如果原来该磁道基本区记录已经放满，则根据磁道索引项指示位置插入新记录后，把最后的溢出记录（具有最大关键码）移出磁道基本区，再根据溢出索引项将这个溢出记录放入溢出区，并以有序链表插入算法将溢出记录链入。



动态索引结构



10.4.2 动态的 m 路搜索树

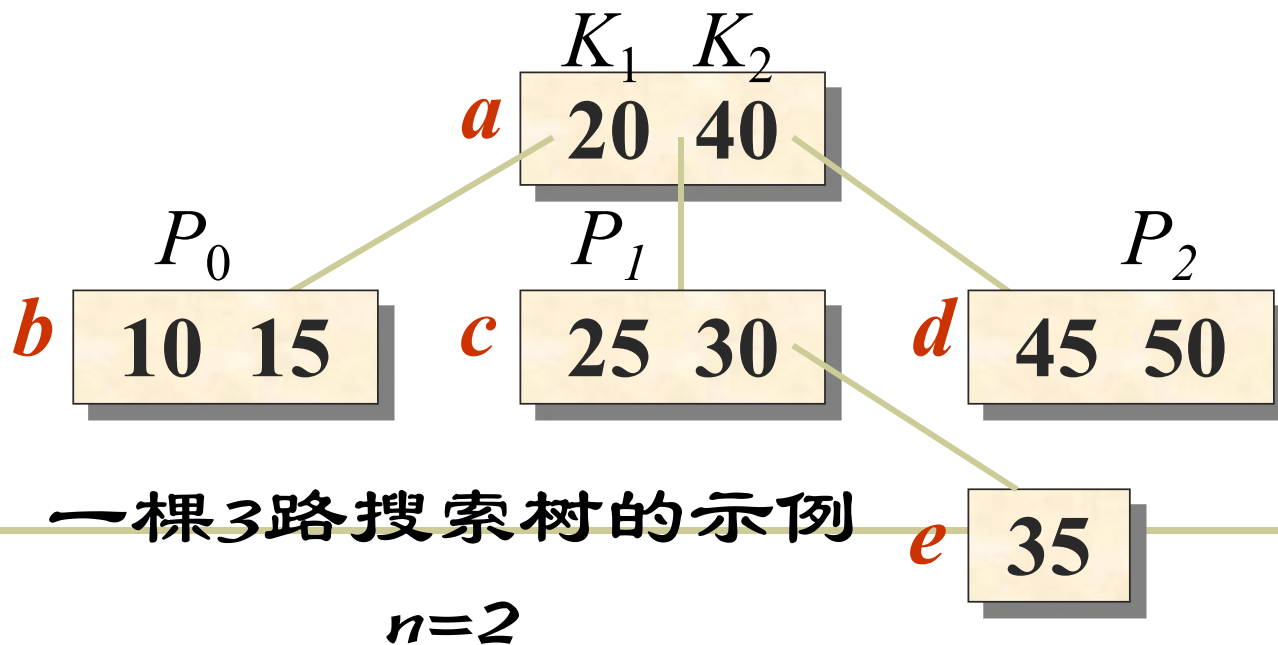
- m 路搜索树多为可以动态调整的多路搜索树，它的递归定义为：
- 一棵 m 路搜索树，它或者是一棵空树，或者是满足如下性质的树：
 - ✓ 根最多有 m 棵子树，并具有如下的结构：

$$(n, P_0, K_1, P_1, K_2, P_2, \dots, K_n, P_n)$$

其中, P_i 是指向子树的指针, $0 \leq i \leq n < m$; K_i 是关键码, $1 \leq i \leq n < m$ 。 $K_i < K_{i+1}$, $1 \leq i < n$ 。



- ✓ 在子树 P_i 中所有的关键码都小于 K_{i+1} , 且大于 K_i , $0 < i < n$ 。
- ✓ 在子树 P_n 中所有的关键码都大于 K_n ;
- ✓ 在子树 P_0 中的所有关键码都小于 K_1 。
- ✓ 子树 P_i 也是 m 路搜索树, $0 \leq i < n$ 。





M路搜索树的C++描述



```
const int MaxValue = .....
```

//关键码集合中不可能有的最大值

```
template <class T>
```

```
struct MtreeNode {
```

//树结点定义

```
    int n;
```

//索引项个数

```
    MtreeNode<T> *parent;
```

//父结点指针

```
    T key[m+1];
```

//key[m]为监视哨，key[0]未用

```
    int *recptr[m+1];
```

//索引项记录起始地址指针

```
    MtreeNode<T> *ptr[m+1];
```

//子树结点指针，ptr[m]
在插入溢出时使用

```
};
```



template <class T>

struct Triple {

MtreeNode<T> *r;

int i;

int tag;

};

//搜索结果三元组

//结点地址指针

//结点中关键码序号i

//tag=0, 成功; =1, 失败

template <class T>

class Mtree {

//m叉搜索树定义

protected:

MtreeNode<T> *root;

//根指针

int m;

//路数

public:

Triple<T> Search(const T& x); //搜索

};



- AVL树是 2 路搜索树。
- 若已知 m 路搜索树的度 m 和它的高度 h , 则树中的最大结点个数为:

$$\sum_{i=1}^h m^{i-1} = \frac{1}{m-1} (m^h - 1)$$

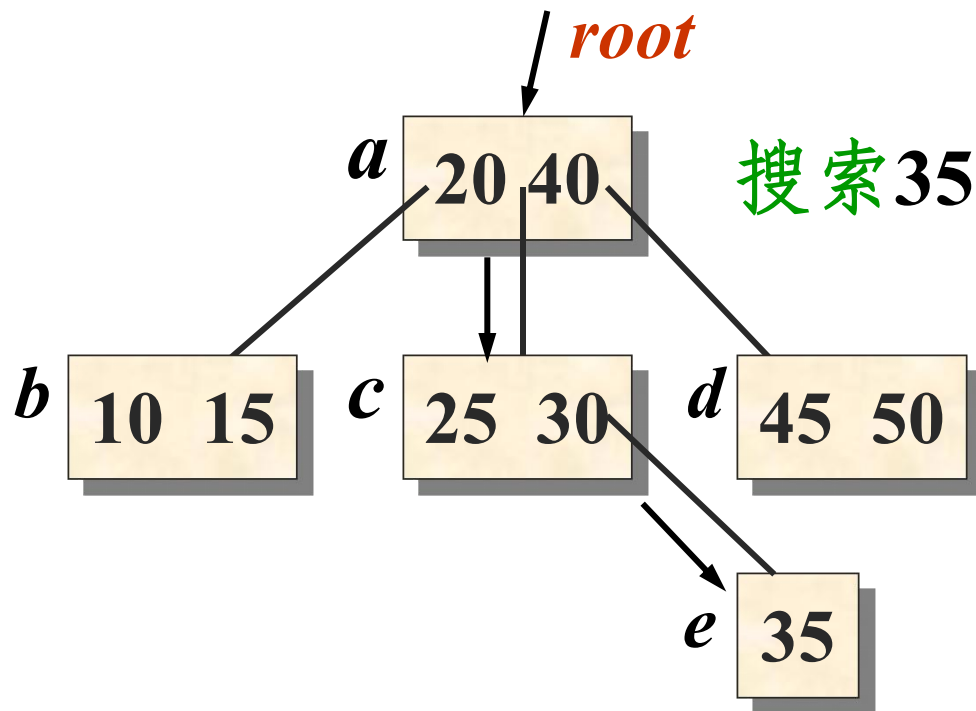
- 每个结点中最多有 $m-1$ 个关键码, 在一棵高度为 h 的 m 路搜索树中关键码最大个数为 $m^h - 1$ 。
 - ✓ 高度 $h=3$ 的二叉搜索树, 关键码最大数为 $2^3 - 1 = 7$;
 - ✓ 高度 $h=4$ 的 3 路搜索树, 关键码最大数为 $3^4 - 1 = 80$ 。



m 路搜索树的搜索算法



- 在 m 路搜索树上的搜索过程是一个在结点内搜索和自根结点向下逐个结点搜索的交替的过程。





```
template <class T>
```

```
Triple<T> Mtree<T>::Search (const T& x) {
```

```
    //用关键码 x 搜索驻留在磁盘上的m路搜索树
```

```
    //各结点格式为n, p[0], (k[1],p[1]),...,(k[n],p[n]), n < m
```

```
    //函数返回一个类型为Triple(r, i, tag)的记录。
```

```
    //tag = 0, 表示 x 在结点r中找到, 该结点的k[i]等于x;
```

```
    //tag = 1, 表示没有找到x, 可插入结点为r, 插入到该
```

```
    //结点的k[i]与k[i+1]之间。
```

```
    Triple result;
```

```
    //记录搜索结果三元组
```

```
    GetNode (root);
```

```
    //从盘上读取结点root
```

```
    MtreeNode<T> *p = root, *q = NULL;
```

```
    //p是扫描指针,q是p的父结点指针
```

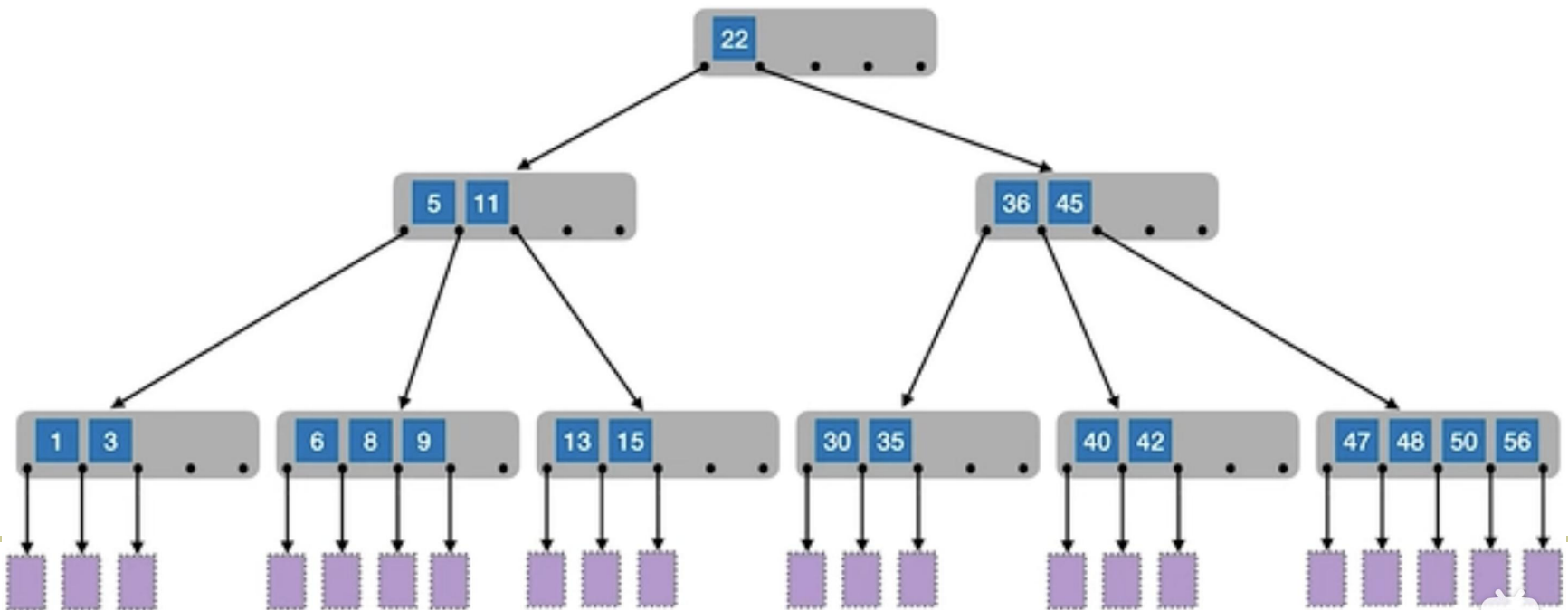
```
    int i = 0;
```



```
while (p != NULL) {                                //从根开始检测
    i = 0; p->key[(p->n)+1] = MaxValue;
    while (p->key[i+1] < x) i++; //在结点内搜索
    if (p->key[i+1] == x) {                          //搜索成功
        result.r = p; result.i = i+1; result.tag = 0;
        return result;
    }
    q = p; p = p->ptr[i];
    //本结点无x, q记下当前结点, p下降到子树
    GetNode(p);                                     //从磁盘上读取结点p
}
result.r = q; result.i = i; result.tag = 1;
return result; //搜索失败, 返回插入位置
};
```




- 提高搜索树的路数 m , 可以改善树的搜索性能。对于给定的关键码数 n , 如果搜索树是平衡的, 可以使 m 路搜索树的性能接近最佳。
- 下面讨论一种称之为B树的平衡的 m 路搜索树。





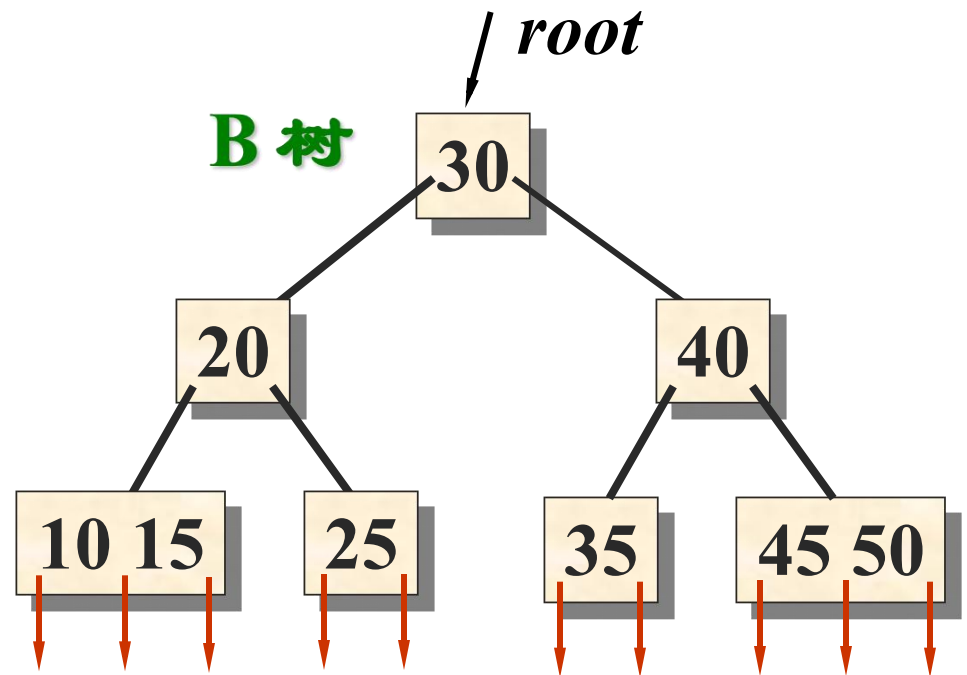
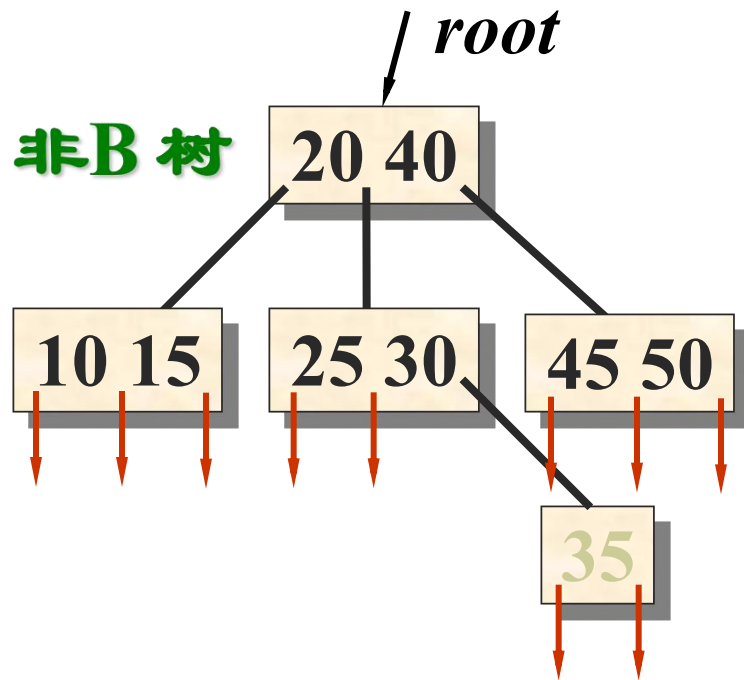
10.4.3 B 树



- 一棵 m 阶B 树是一棵平衡(balanced)的 m 路搜索树, 它或者是空树, 或者是满足下列性质的树:
 - ✓ 根结点至少有 2 个子女。
 - ✓ 除根结点以外的所有结点 (不包括失败结点) 至少有 $\lceil m/2 \rceil$ 个子女。
 - ✓ 所有的失败结点都位于同一层。
- 在B 树中的“失败”结点是当 x 不在树中时才能到达的结点。这些结点实际不存在, 指向它们的指针为NULL。它们不计入树的高度。



- 注意， m 阶B树继承了 m 路搜索树的定义。原来 m 路搜索树定义中的规定，在 m 阶B树中都保留。
- 事实上，在B树的每个结点中还包含有一组指针 $\text{recptr}[m+1]$ ，指向实际记录的存放地址。
- $\text{key}[i]$ 与 $\text{recptr}[i]$ ($1 \leq i \leq n < m$) 形成一个索引项 $(\text{key}[i], \text{recptr}[i])$ ，通过 $\text{key}[i]$ 可找到某个记录的存储地址 $\text{recptr}[i]$ 。
- 在讨论B树结构的操作时先不涉及 $\text{recptr}[i]$ ，因此在后续讨论中该指针不出现。





B树类和B树结点类的定义



```
template <class T>
```

```
class Btree : public Mtree<T> {           //B树类定义
```

```
    //继承m叉搜索树的所有属性和操作，
```

```
    //Search从Mtree继承， MtreeNode直接使用
```

```
public:
```

```
    Btree();                               //构造函数
```

```
    bool Insert (const T& x);              //插入关键码x
```

```
    bool Remove (T& x);                    //删除关键码x
```

```
};
```



B 树的搜索算法



- B树的搜索算法继承了 m 路搜索树Mtree上的搜索算法。
- B树的搜索过程是一个在结点内搜索和循某一条路径向下一层搜索交替进行的过程。
 - 搜索成功，报告结点地址及在结点中的关键码序号；
 - 搜索不成功，报告最后停留的叶结点地址及新关键码在结点中可插入的位置。
- B 树的搜索时间与B 树的阶数 m 和B 树的高度 h 直接有关, 必须加以权衡。



- 在**B** 树上进行搜索, 搜索成功所需的时间取决于关键码所在的层次; 搜索不成功所需的时间取决于树的高度。
- 定义**B**树的高度 h 为叶结点 (失败结点的双亲) 所在的层次, 那树的高度 h 与树中的关键码个数 N 之间有什么关系?
- 如果让**B**树每层结点个数达到最大 ($m-1$), 且设关键码总数为 N , 则树的高度达到最小:

$$N \leq m^h - 1 \rightarrow h \geq \lceil \log_m(N+1) \rceil$$



- 如果让 m 阶B树中每层结点个数达到最少，则B树的高度可能达到最大。设树中关键码个数为 N ，从B树的定义知：
 - ✓ 1层：1个结点
 - ✓ 2层：至少2个结点
 - ✓ 3层：至少 $2\lceil m/2 \rceil$ 个结点
 - ✓ 4层：至少 $2\lceil m/2 \rceil^2$ 个结点如此类推，.....
 - ✓ h 层：至少有 $2\lceil m/2 \rceil^{h-2}$ 个结点。
- 所有这些结点都不是失败结点。失败结点在第 $h+1$ 层，失败结点个数为 $N+1$ 。



- 这是因为树中关键码有 N 个，而失败数据一般与已有关键码交错排列。因此，有
 - ✓ $N+1 = \text{失败结点数} = \text{位于第 } h+1 \text{ 层的结点数}$
$$\geq 2 \lceil m/2 \rceil^{h-1}$$
$$\therefore N \geq 2 \lceil m/2 \rceil^{h-1} - 1$$
$$\therefore h-1 \leq \log_{\lceil m/2 \rceil} ((N+1)/2)$$
$$\therefore h \leq \log_{\lceil m/2 \rceil} ((N+1)/2) + 1$$
- 示例：若B 树的阶数 $m = 199$, 关键码总数 $N = 1999999$, 则B 树的高度 h 不超过

$$\log_{100} 1000000 + 1 = 4$$



m 值的选择



- 如果提高B 树的阶数 m , 可以减少树的高度, 从而减少读入结点的次数, 因而可减少读磁盘的次数。
- 事实上, m 受到内存可使用空间的限制。当 m 很大, 超出内存工作区容量时, 结点不能一次读入到内存, 增加了读盘次数, 也增加了结点内搜索的难度。
- m 值的选择: 应使得在B 树中找到关键码 x 的时间总量达到最小。



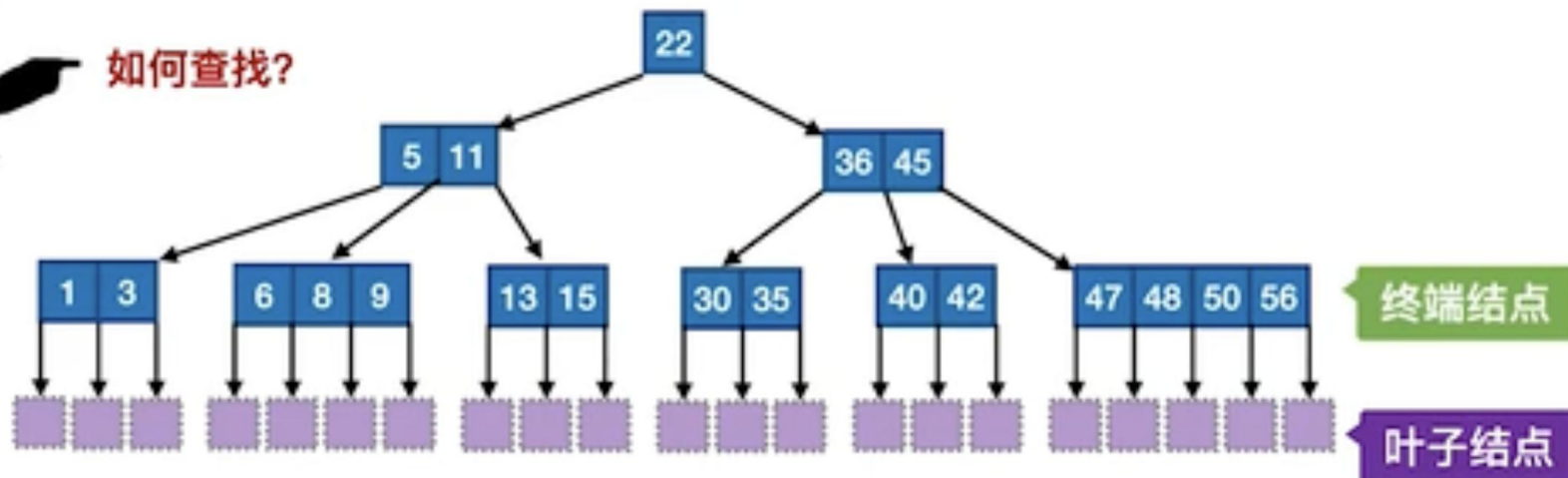
- 这个时间由两部分组成：
 - ✓ 从磁盘中读入结点所用时间
 - ✓ 在结点中搜索 x 所用时间
- 根据定义, B 树的每个结点的大小都是固定的, 结点内最多有 $m-1$ 个索引项 $(key_i, recptr_i, P_i)$, $1 \leq i < m$ 。
- 设 key_i 所占字节数为 α , $recptr_i$ 和 P_i 所占字节数为 β , 则结点大小近似为 $m(\alpha+2\beta)$ 个字节。读入一个结点所用时间为:

$$t_{seek} + t_{latency} + m(\alpha+2\beta) t_{tran} = a + bm$$



如何查找?

字不重要，
看图！



m阶B树的核心特性:

尽可能“满”

1) 根节点的子树数 $\in [2, m]$, 关键字数 $\in [1, m-1]$ 。

其他结点的子树数 $\in [\lceil m/2 \rceil, m]$; 关键字数 $\in [\lceil m/2 \rceil - 1, m-1]$

尽可能“平衡”

2) 对任一结点, 其所有子树高度都相同

3) 关键字的值: 子树0 < 关键字1 < 子树1 < 关键字2 < 子树2 < (类比二叉查找树 左 < 中 < 右)

含n个关键字的m叉B树, $\log_m(n+1) \leq h \leq \log_{\lceil m/2 \rceil} \frac{n+1}{2} + 1$



10.4.4 B 树的插入



- B 树是从空树起, 逐个插入关键码而生成的。
- 在B 树中每个非失败结点的关键码个数都在
$$[\lceil m/2 \rceil - 1, m-1]$$
之间。
- 插入在某个叶结点开始。如果在关键码插入后结点中的关键码个数超出了上界 $m-1$, 则结点需要“分裂”, 否则可以直接插入。
- 实现结点“分裂”的原则是:
 - ✓ 设结点 p 中已经有 $m-1$ 个关键码, 当再插入一个关键码后结点中的状态为



$(m, P_0, K_1, P_1, K_2, P_2, \dots, K_m, P_m)$

其中 $K_i < K_{i+1}, 1 \leq i < m$

- ✓ 这时必须把结点 p 分裂成两个结点 p 和 q , 它们包含的信息分别为:

➤ 结点 p :

$(\lceil m/2 \rceil - 1, P_0, K_1, P_1, \dots, K_{\lceil m/2 \rceil - 1}, P_{\lceil m/2 \rceil - 1})$

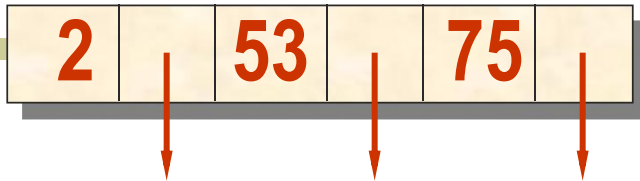
➤ 结点 q :

$(m - \lceil m/2 \rceil, P_{\lceil m/2 \rceil}, K_{\lceil m/2 \rceil + 1}, P_{\lceil m/2 \rceil + 1}, \dots, K_m, P_m)$

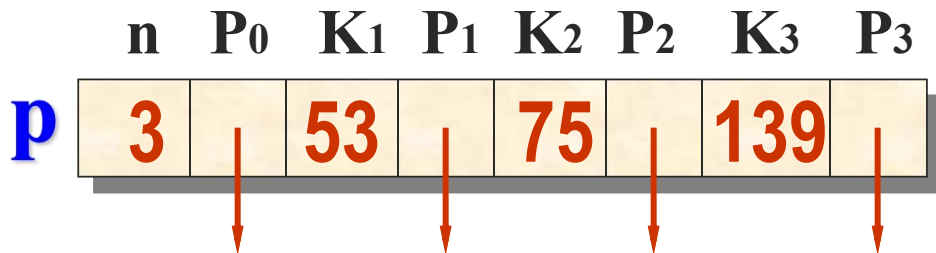
- ✓ 位于中间的关键码 $K_{\lceil m/2 \rceil}$ 与指向新结点 q 的指针形成一个二元组 $(K_{\lceil m/2 \rceil}, q)$, 插入到这两个结点的双亲结点中去。



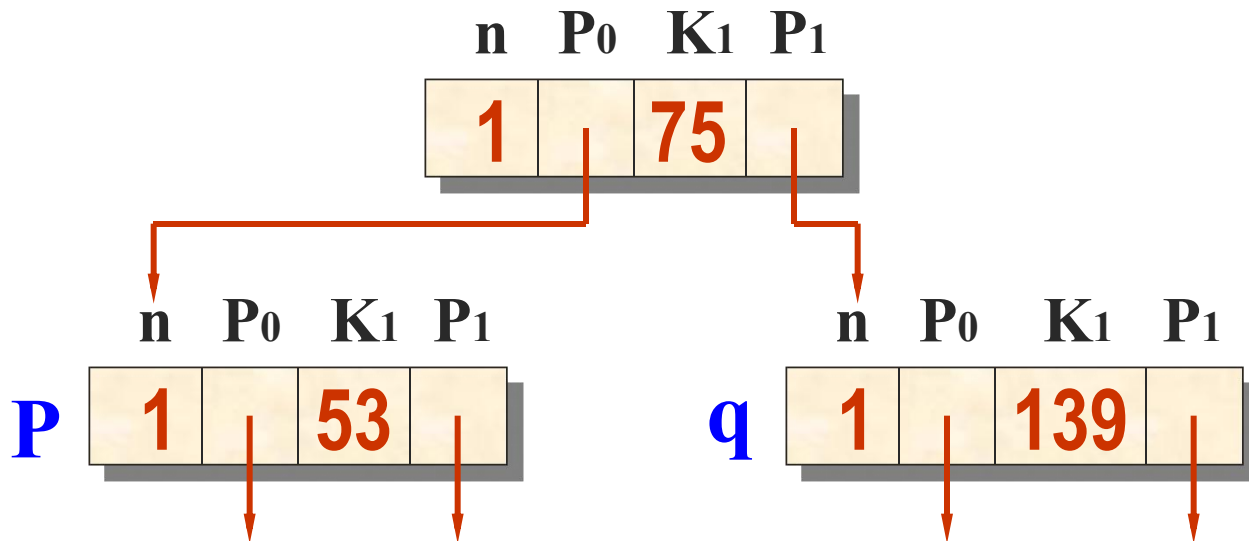
p n P₀ K₁ P₁ K₂ P₂ m = 3



加入139,
结点溢出



结点
分裂



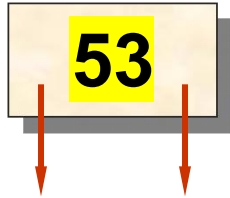
结点“分裂”的示例



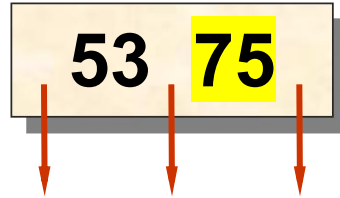
示例:从空树开始加入关键码建立3阶B树



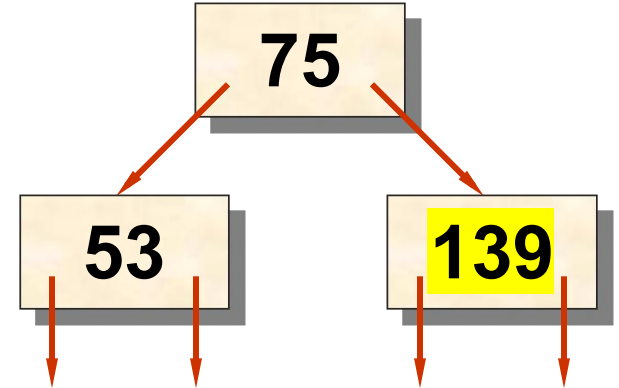
n=1 加入53



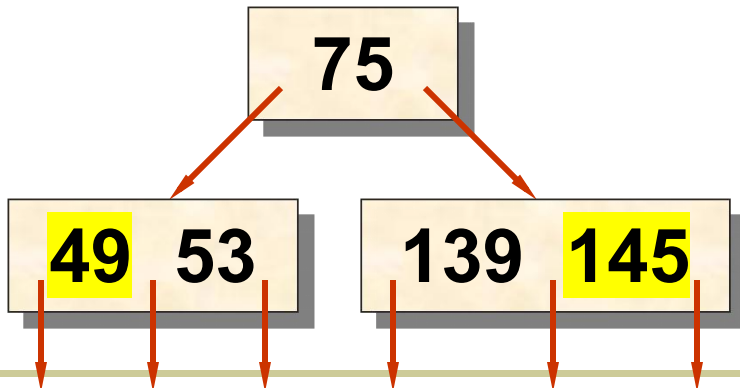
n=2 加入75



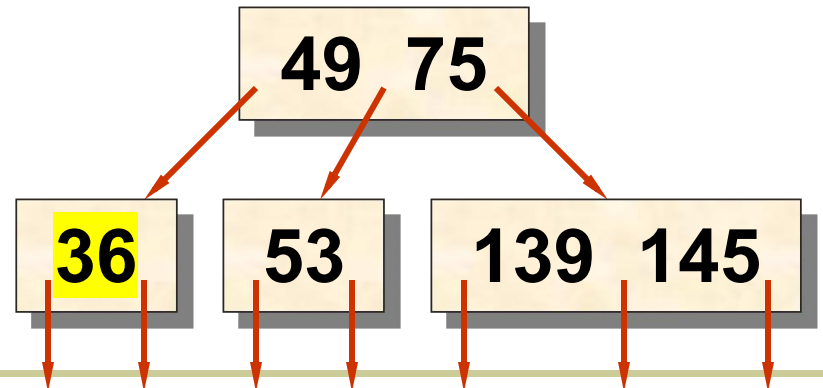
n=3 加入139



n=5 加入49,145

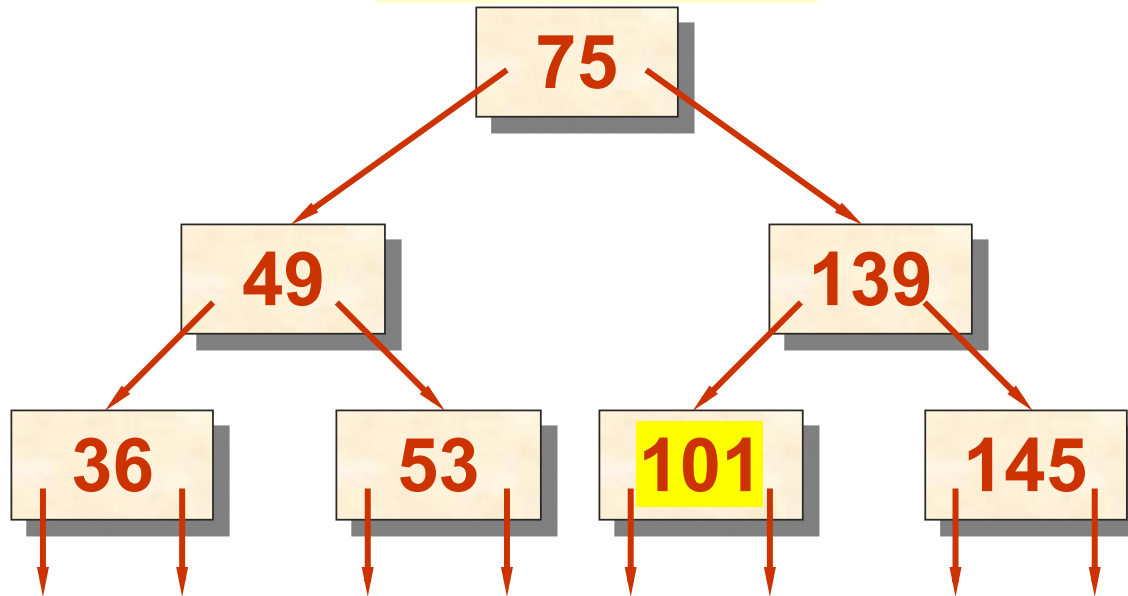


n=6 加入36





$n=7$ 加入 101



若设B 树的高度为 h ，则在自顶向下搜索到叶结点的过程中需要进行 h 次读盘。

- 在插入新关键码时，需要自底向上地分裂结点，最坏情况下从被插关键码所在叶结点到根的路径上的所有结点都要分裂。